

Transformer: Attention Is All You Need

Abstract

- 기존의 시퀀스 변환 모델
 - 보통 인코더-디코더 구조를 포함한 RNN, CNN 기반
 - SOTA는 attention을 이용해 encoder와 decoder를 연결
- Transformer
 - 어텐션만을 사용하여 기계번역 태스크에서 타 모델에 비해 뛰어난 성능 기록
 - 더 높은 성능을 가지면서 병렬화가 쉽고 학습시간이 적음
 - WMT 2014 English-to-German translation
 - 28.4 BLEU
 - 기존 SOTA(ensemble)보다 2 BLEU 이상 향상
 - WMT 2014 English-to-French translation
 - 8개의 GPU로 3.5일 학습으로 41.8 BLEU
 - 단일 모델 기준 SOTA
 - 다른 task 일반화도 잘 됨

1. Introduction

이 논문의 기술적 배경

- RNN, LSTM, GRU는 언어 모델링이나 기계번역 task 등 sequence modeling에서 좋은 모델로 자리잡아 있음

- 이후 많은 recurrent 모델과 encoder-decoder 구조의 성능이 발전해 옴
- Recurrent model
 - 이전의 hidden state h_{t-1} 과 t 에서의 input을 받아 현재의 hidden state h_t 를 만들어냄
 - 순차적 특성으로 인해 병렬 처리가 불가능 + 시퀀스가 길어질수록 메모리 제약 등 문제
- 최근 연구
 - factorization과 conditional computation으로 연산 효율성 높임
 - conditional computation의 경우 모델 성능까지 개선
 - but 여전히 순차적 계산이라는 근본적 문제는 해결하지 못함

이 논문이 제안하는 기술

- Transformer
 - attention만을 기반으로 하는 새로운 간단한 네트워크 구조
 - RNN, CNN이 전혀 필요 x
- global dependencies
 - attention으로 떨어진 거리와 관계 없이 dependency를 모델링 가능
- 훨씬 높은 수준의 병렬화
 - P100 GPU 8개로 12시간의 학습만으로 기계 번역에서 최고의 성능

2. Background

Extended Neural GPU, ByteNet, ConvS2S

- 순차적 계산을 줄이려는 노력
- CNN을 기본으로 하며 모든 입력/출력에 대하여 은닉 표현을 병렬적으로 계산
- 임의의 두 위치를 연결하는 데에 드는 연산량이 거리에 따라 증가

- ConvS2S → linearly increasing, ByteNet → logarithmically increasing
- **transformer에서는 거리에 상관없이 같은 연산량이 필요!**
 - but 어텐션 가중합을 이용하므로 효과적 해상도가 감소
 - → 이후 multi-head attention으로 해결

Self-attention

- 단일 시퀀스 내 서로 다른 위치 간 관계를 학습하여 시퀀스의 표현을 계산하는 어텐션 메커니즘
- 독해, 요약, textual entailment, learning task-independent sentence representations 등에서 성공을 이루어 냄

End-to-End Memory Networks

- RNN이 아닌 recurrent attention mechanism을 기반
- 간단한 질문응답(simple QA)이나 언어 모델링 작업에서 좋은 성능

Transformer는 RNN/CNN 없이 self-attention만을 사용하며 RNN/CNN 없이 입력과 출력의 표현을 계산하는 첫번째 모델!!

3. Model Architecture

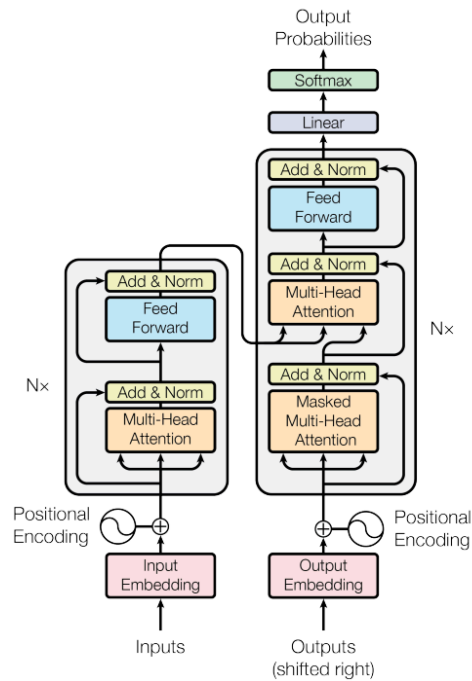
Encoder

- input sequence ($x_1, \dots, x_n$)로 연속적인 표현 $z = (z_1, \dots, z_n)$ 을 만들어 냄

Decoder

- $z = (z_1, \dots, z_n)$ 을 입력으로 받아 output sequence ($y_1, \dots, y_m$)를 한번에 하나씩 순차적으로 생성

- 다음 심볼을 생성할 때 이전에 생성된 심볼들을 추가 입력으로 사용하는 자동 회귀 (auto-regressive) 사용
- 인코더와 디코더 모두에서 stacked self-attention과 point-wise, fully connected layers를 사용



3.1. Encoder and Decoder Stacks

Encoder

- N=6개의 같은 layer
 - 각 layer는 두 개의 sub-layers
 1. multi-head self-attention
 2. FC feed-forward
- 두 sub-layer에서 residual connection을 적용 후 layer normalization 적용

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- 원활한 잔차연결의 위해 모든 sublayer와 embedding layer에서 차원을 $d_{\text{model}}=512$ 로 유지

Decoder

- N=6개의 같은 layer
 - 인코더 스택 출력에 대한 multi-head self-attention이 추가되어 3개의 sublayers
 1. Masked Self-Attention
 2. Encoder-Decoder Attention
 3. Feed Forward
- 각 sub-layer에서 residual connection을 적용 후 layer normalization 적용

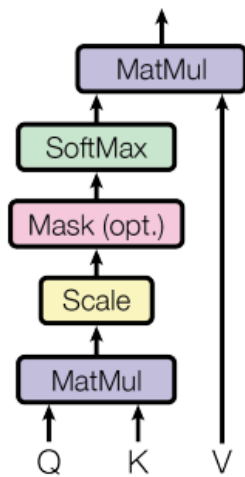
$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- 디코더 스택의 multi-head attention은 이후 위치를 참조하지 못하도록 masked 되어 있음
- 디코더 입력 문장이 한칸씩 오프셋 되어있기에 i의 예측은 오직 이전 위치(< i)의 출력에만 의존
- **인코더 정보 + 이전 출력 정보**를 조합하여 순차적으로 출력을 생성

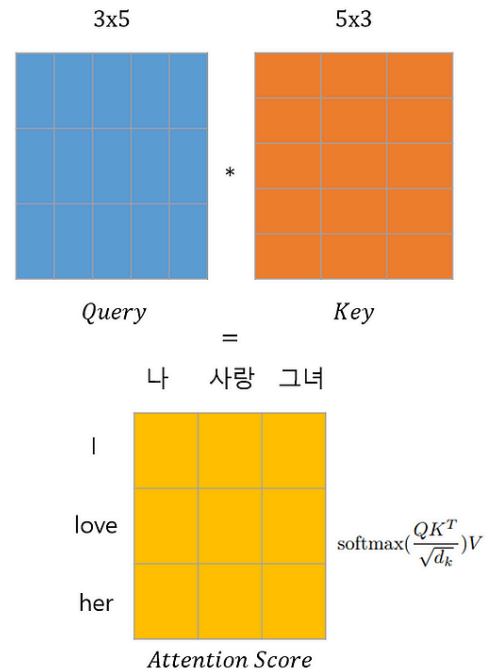
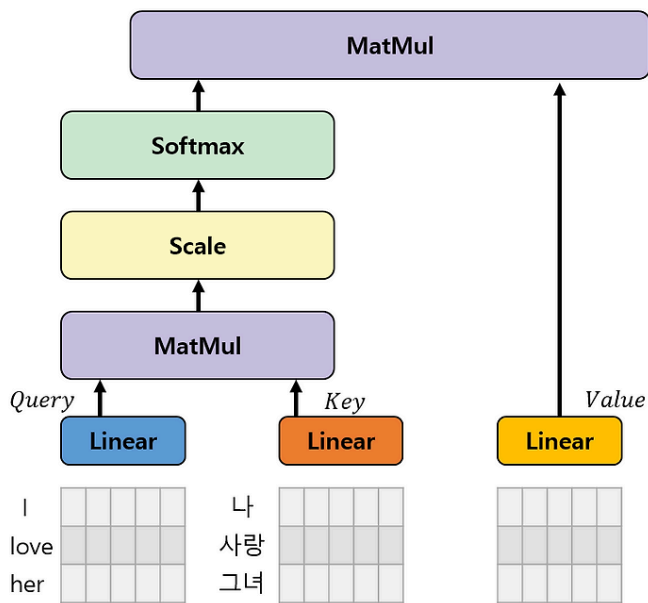
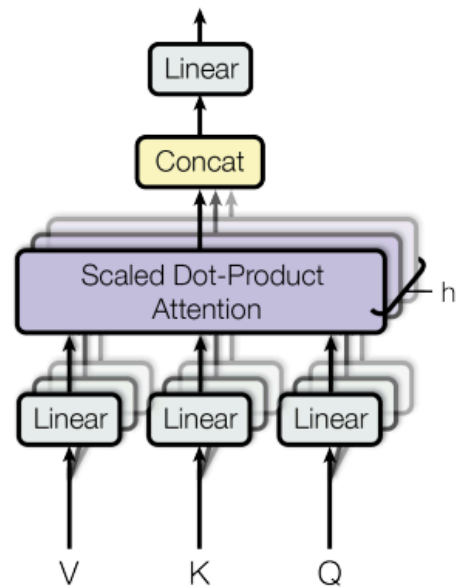
3.2. Attention

- 쿼리(query) 와 키-값(key-value) 쌍들의 집합을 출력으로 매핑하는 함수
- 쿼리와 키 사이의 유사도 확률분포와 value와의 가중합으로 계산

Scaled Dot-Product Attention



Multi-Head Attention



3.2.1. Scaled Dot-Product Attention

- Query + 여러개의 K, V → output
- query를 모든 key들과 곱한뒤 각각 루트 d_k 로 나눔
 - d_k : V의 차원

→ softmax를 적용하여 확률분포를 구한 뒤 value와 가중합

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- 가장 많이 사용되는 어텐션 연산
 - dot-product attention: scaling factor를 제외하면 위 식과 같음
 - additive attention: 피드포워드 신경망(은닉층 하나 포함) 을 사용해 쿼리와 키의 호환 함수를 계산함
- 복잡도는 비슷하나 dot-product가 최적화된 행렬곱 계산을 사용하기 때문에 더 빠르고 메모리적 효율이 큼
- d_k 값이 작은 경우에는 성능이 비슷하나, d_k가 클 경우 additive attention이 scaling이 없는 dot-product attention보다 성능이 좋음
- d_k가 커질수록 dot-product가 더 커지면서 softmax함수에서 기울기가 0에 가까운 부분에 도달하기 때문
 - scaled dot-product 사용

Multi-Head Attention

- d_model 차원의 Q,K,V에 대해 하나의 attetion 함수를 적용하는 것 보다 각각을 d_k, d_k, d_v 차원으로 선형 변환을 학습된 서로 다른 선형 변환을 통해 h번 반복하는 것이 더 효과적
- 각 Q, K, V 값에 대해 어텐션 함수를 병렬로 수행 → 각 어텐션에서 d_v 차원의 출력이 나오고 h개의 출력을 concat하여 다시 한번 선형변환을 거침
- 여러 개로 나눈 Multi-Head들이 서로 다른 Representation Subspaces를 학습하여 다양한 유형의 종속성을 가져 다양한 정보를 결합 가능
 - 단일 어텐션은 평균내어버려 위 특성이 사라짐
- 각 어텐션은 문장 타입, 명사, 관계, 강조 등 집중하는 부분이 다르기에 더 풍부한 정보 학습 가능

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

- 이 논문에서는 $h=8$ 로 설정하였고 $d_k=d_v=d_{\text{model}}/h=64$ 임
- 각 어텐션 헤드의 차원이 축소되어 단일 어텐션을 d_{model} 에서 진행하는 것과 비슷한 계산량을 가짐

Applications of Attention in our Model

1. encoder-decoder attention layer
 - Q: 이전 디코더 층의 출력에서 가져옴
 - K,V: 인코더의 출력에서 가져옴
 - 이렇게 하면 디코더의 각 위치가 입력 시퀀스 전체를 바라볼 수 있음
 - 기존 seq2seq 모델([Bahdanau et al. 2014] 등)에서 사용된 전통적인 인코더-디코더 어텐션 구조를 그대로 모방
2. 인코더의 self-attention layer
 - Q,K,V: 모두 인코더의 출력
 - 인코더의 각 위치는 인코더 이전 층의 모든 위치에 주의를 기울일 수 있음
 - 입력 시퀀스 내에서 단어들 간의 **전역적인 의존 관계**를 모델링
3. 디코더의 self-attention layer
 - Q,K,V: 모두 디코더의 출력
 - 2번과 비슷하나 masking 되어 있어 디코더의 각 위치는 자기 자신 이전의 위치만 참고 가능
 - 미래 단어를 미리 보면 안 되고 과거 단어들에만 기반해 다음 단어를 예측(자동 회귀)
 - 미래 위치에 해당하는 값들을 softmax 입력에서 $-\infty$ 로 마스킹하여 미래 위치 참조를 막음

Position-wise Feed-Forward Networks

- attention sublayer 이외에도 Fully connected feed-forward network가 존재
- 각 위치에 독립적이고 동일하게 적용
- 두 개의 선형 변환과 ReLU 함수로 구성됨

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- 첫 번째 선형 변환 $W_1, b_1 \rightarrow$ 입력을 더 큰 차원으로 확장 (비선형성 적용)
- ReLU $\rightarrow$ 음수는 0으로, 양수만 통과
- 두 번째 선형 변환 $W_2, b_2 \rightarrow$ 다시 원래 차원으로 축소
- 층마다 가중치 W, b 가 달라짐
- 커널 크기가 1인 두개의 convolution으로 생각해도 됨
- $d_{\text{model}}=512$, 내부 layer의 차원은 $d_{\text{ff}}=2048$

512차원 $\rightarrow$ (확장) 2048차원 $\rightarrow$ (축소) 512차원

Embeddings and Softmax

- 다른 시퀀스 변환 모델과 마찬가지로 입력 토큰과 출력 토큰을 d_{model} 차원의 벡터로 변환하기 위해 학습된 임베딩 사용
- 디코더의 출력을 예측한 다음 토큰 확률 값으로 변환하기 위해 선형변환과 softmax 함수를 사용
- 두 embedding layer(입력, 출력 임베딩)와 pre-softmax 선형변환에서 같은 가중치 벡터를 사용함

-embedding layer에서는 루트 d_{model} 로 scaling

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Positional Encoding

- RNN이나 CNN을 사용하지 않으므로 상대적, 절대적 위치 정보를 표현 가능한 positional encoding 이 필요
- 임베딩과 동일한 d_{model} 의 차원을 가지고 있어 더하기 연산 가능
- 여러가지 형태의 positional encoding이 존재하나 이 논문에서는 sin, cos 함수를 사용

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- pos: 토큰의 위치 (0, 1, 2, ...)
- i: 임베딩 차원 인덱스
- 임베딩의 각 차원은 다른 주파수를 가진 사인/코사인 파형으로 표현
- 파장의 길이는 2π 에서 $10000 \cdot 2\pi$ 까지 기하급수적으로 늘어납니다.

sinusoidal을 선택한 이유

- 모델이 상대적인 위치 관계를 쉽게 학습할 수 있음
- 특정 위치에서 k만큼 떨어진 위치의 인코딩은 선형 변환으로 표현 가능
- 학습된 임베딩과 거의 비슷한 결과를 내었고 훈련 시 본 적 없는 더 긴 시퀀스에도 일반화될 수 있다는 장점

Why Self-Attention

세 가지의 기준을 가지고 transformer를 기존 RNN/CNN과 비교

1. Computational Complexity (계산 복잡도)

- 각 레이어의 계산량

2. Parallelization (병렬화 가능성)

- 순차적으로 처리해야 하는 최소 단계 수

3. Path Length (경로 길이)

- 입력과 출력 사이에서 정보가 오가는 경로의 길이
- 짧을수록 long-range dependencies를 잘 학습할 수 있음
- 각 레이어의 입력 및 출력에서 가장 경로가 긴 길이를 비교

RNN과의 비교

- Table 1에서 제시하였듯 순차적 계산에 있어 RNN은 $O(n)$ 의 시간 복잡도를 가지는 반면 self-attention은 $O(1)$ 의 시간 복잡도를 가짐
- 각 레이어의 계산량은 시퀀스 길이 n 이 차원 d 보다 작을 경우 self-attention이 RNN에 비해 빠름 (대부분의 경우 word-piece, byte-pair)

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

- 문장의 길이가 매우 길 경우 전체 시퀀스를 보지 않고 r 의 크기의 window 내의 단어만 보도록 제한하는 방법으로 개선 가능 → 최대 경로 길이가 $O(n/r)$ 로 늘어남

CNN과의 비교

- 단일 CNN 레이어 (커널 크기 $k < n$):
 - 모든 입력-출력을 한 번에 연결 불가.
 - 전체 연결을 만들려면 여러 층을 쌓아야 함:
 - 연속 커널(Contiguous kernel): $O(n/k)$ 층 필요
 - Dilated convolution: $O(\log_k(n))$ 층 필요
 → 따라서 입력과 출력 사이의 경로가 길어짐.
- 비용(Complexity):
 - CNN은 보통 RNN보다 k 배 비쌈.
 - 하지만 Separable Convolution을 쓰면 비용이 줄어서:
 - $O(k \cdot n \cdot d + n \cdot d^2)$ 로 계산 가능.
 - 그래도 $k = n$ 일 경우:
 - Separable Convolution의 비용 = Self-Attention + Point-wise Feed-Forward Layer 비용과 같음.
 - Transformer가 사용하는 방식(Self-Attention + FFN)이랑 동일해짐.

추가 장점

- 어텐션 헤드마다 다른 역할을 배워 표현력이 좋아짐

Training

Training Data and Batching

- 450만개 문장 쌍을 가지는 WMT 2014 English-German dataset 사용
- byte-pair 방식으로 토큰화하여 37,000개 토큰의 단일 vocabulary
- 3600만개의 문장 쌍을 가지는 WMT 2014 English-French dataset 사용
- word-piece 방식으로 토큰화하여 25000 소스 토큰 + 25000 타겟 토큰

Hardware and Schedule

- 8개의 P100 GPU 이용
- 학습 스텝 당 0.4초, 총 100000스텝(12시간) 학습
- big model에서는 학습 스텝 당 1초, 총 300000스텝(3.5일) 학습

Optimizer

- Adam optimizer ($B1=0.9$, $B2=0.98$, $\epsilon=10^{-9}$)
- 첫 `warmup_steps`에서는 `lr`을 선형적으로 증가시켰다가 스텝 수의 루트 역수만큼씩 감소
- `warmup_steps=4000`

$$lr_{rate} = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

Regularization

- BLEU: n-gram 기반
- FLOPs: Floating point Operations

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Residual Dropout

- 각 sublayer에 드롭아웃 추가
- 인코더와 디코더에서, **임베딩 + 위치 인코딩의 합**에도 Dropout 적용.
- Base 모델에서 Dropout 비율 **Pdrop = 0.1** 사용.

Label Smoothing

- label smoothing $\epsilon_{ls} = 0.1$ 로 설정
- 과적합 막음
- Perplexity(언어 모델 지표)는 약간 나빠지지만, 정확도와 BLEU 점수는 향상

Results

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

Machine Translation (기계 번역)

-WMT 2014 English-to-German translation task

- transformer (big)는 이전의 최고모델에 비해 BLEU 점수가 2.0 이상 높았음 $\Rightarrow$ 28.4 (new SOTA)
- transformer (big) 학습 시간: 3.5일
- Base 모델조차도 기존 모든 모델·양상블을 능가하면서도 훨씬 적은 학습 비용

-WMT 2014 English-to-French translation task

- transformer (big)의 BLEU 점수는 41.0으로 기존 단일 모델을 능가함
- 학습 비용은 기존 SOTA의 1/4 이하
- 영어 $\rightarrow$ 프랑스어 big 모델은 dropout = 0.1 사용 (독일어 big 모델은 0.3).

-Checkpoint Averaging

- Base 모델: 마지막 5개 체크포인트(10분 간격 저장)를 평균
- Big 모델: 마지막 20개 체크포인트 평균

-추론 시 설정

- Beam search: 빔 크기 4, 길이 패널티 $\alpha = 0.6$
- 최대 출력 길이 = 입력 길이 + 50
- 가능할 경우 조기 종료 (early stopping)

-비용 추정

- Table 2에서 다른 모델과 번역 품질 및 학습 비용 비교
- 학습 비용(FLOPs) 추정 = 학습 시간 × GPU 개수 × GPU 별 부동소수점 성능

Model Variations

- Transformer 구성 요소의 중요성 검증
 - 영어 → 독일어 dev set(newstest2013)에서 다양한 변형 실험 수행
- Beam search 사용, 체크포인트 평균은 사용하지 않음.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$	
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65	
(A)					1	512	512				5.29	24.9	
					4	128	128				5.00	25.5	
					16	32	32				4.91	25.8	
					32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58	
					32					5.01	25.4	60	
(C)	2									6.11	23.7	36	
	4									5.19	25.3	50	
	8									4.88	25.5	80	
	256				32	32				5.75	24.5	28	
	1024				128	128				4.66	26.0	168	
			1024								5.12	25.4	53
			4096								4.75	26.2	90
(D)							0.0			5.77	24.6		
							0.2			4.95	25.5		
								0.0		4.67	25.3		
								0.2		5.47	25.7		
(E)	positional embedding instead of sinusoids									4.92	25.7		
big	6	1024	4096	16				0.3	300K	4.33	26.4	213	

- (A): 어텐션 헤드 개수, 키/밸류의 차원
 - 단일 헤드: 최고 모델에 비해 BLEU 0.9 낮음
 - 헤드가 너무 많아도 성능 떨어짐
- (B): 어텐션 키 차원인 d_k 를 낮추니 성능 저하
 - 호환성(compatibility) 계산이 단순하지 않음
 - 단순 dot-product 대신 더 정교한 함수가 도움 될 수 있음
- (C), (D): 모델은 클수록 성능이 좋고 드롭아웃은 오버피팅을 방지하는 데에 효과적
- (E): sinusoidal 대신 사전 학습된 positional embedding 사용
 - 비슷한 성능

English Constituency Parsing (영어 구문 분석)

- Transformer가 번역 외 다른 과제에도 일반화 가능한지 검증
- 구문 분석(Constituency Parsing): 입력에 비해 출력이 길고, 강한 구조 제약이 있는 과제
- 기존 RNN seq2seq 모델은 데이터가 적은 경우 SOTA 달성 실패
- 훈련 설정
 - WSJ Penn Treebank (약 40K 문장)에서 4층Transformer, $d_{\text{model}}=1024$ 학습
 - semi-supervised 설정에서는 BerkeleyParser 코퍼스 + high-confidence 데이터 (~1,700만 문장) 사용
 - 어휘 크기: WSJ-only = 16K, semi-supervised = 32K
- 하이퍼파라미터
 - dropout, learning rate, beam size 등 최소한의 튜닝만 Section 22 development set(Section 22)에서 수행.
 - 다른 설정은 영어 → 독일어 base 모델 그대로 유지.
 - 추론 시:
 - 최대 출력 길이 = 입력 길이 + 300.
 - Beam size = 21, $\alpha = 0.3$.

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

- Task-specific 튜닝 없이도 Transformer가 매우 좋은 성능 달성
- Recurrent Neural Network Grammar (RNNG) 제외한 모든 이전 모델보다 우수
- 특히 WSJ 40K 데이터만 사용했을 때조차 BerkeleyParser를 능가 → RNN seq2seq 모델과는 대조적

Conclusion

- Transformer ⇒ 최초의 순수 Attention 기반 시퀀스 변환 모델
- 기존의 RNN/CNN 기반 모델에서 벗어나 셀프어텐션만을 활용하여 새로운 SOTA 달성

장점

- 기계 번역 태스크에서 RNN·CNN 기반 아키텍처보다 학습 속도 훨씬 빠름
- 영어 → 독일어, 영어 → 프랑스어에서 새로운 SOTA
- 특히 영어 → 독일어에서는 기존 모든 앙상블 모델보다도 우수

향후 계획

- Attention 기반 모델을 텍스트 외의 모달리티(이미지, 오디오, 비디오)에도 확장
- 긴 입력/출력을 효율적으로 처리하기 위해 국소적(local), 제한적(restricted) Attention 탐구 예정
- 생성 과정을 덜 순차적으로(sequential) 만드는 것도 목표

느낀점

기존의 당연하게 사용되던 RNN, LSTM 구조를 완전히 없애고 새로운 구조를 제안하면서 SOTA를 달성했다는 점에 놀랐다. 단순히 그 구조가 좋았다가 아니라 꼼꼼한 분석을 통해 왜 self attention인지, 등을 명확하게 밝힌 점이 인상깊었다. 앞으로 새로운 논문을 접할 때, 단순히 그렇구나가 아니라 왜 저자가 이 구조를 생각해내게 되었는지, 이 부분을 없애면 생기는 문제들에 초점을 맞추어 분석해야겠다는 생각을 했다.